

Practice with the KIM1 module Kineis satellite constellation

François SPIES – University Marie et Louis Pasteur – Master of Science – Internet of Things – Smart Cities

Necessary hardware for the practice

- one KIM1 Kineis Module to send message to the satellite in low orbit
- one Adafruit Feather LoRa ARM M0 module and one micro-USB cable
- a 1200 mAh or 2400 mAh Lithium-ion battery
- Dupont wires to connect KIM1 to Adafruit feather
- a 400MHz antenna
- a USB multimeter
- Additionally a Lora ARM M0 to receive messages from KIM1/LoRa module

Presentation of the Kineis network between IoT module and the satellite

KIM1 board allows to transmit from an object on the ground/Ocean towards a satellite in LEO (Low Earth Orbit). The satellite altitude is between 500 and 900 km and the footprint is around 6000 km. The time for one loop over the earth is around 1 hour and a half. The orbit is over the north and south poles. In France latitude, there is around 6 to 8 passes each day.

The frequency used is 401.650 MHz using BPSK or QPSK modulation from 200 to 400 Hz.

Each transmission can send up to 31 bytes during around 1 second.

It is possible to receive messages from the satellite, but with another chipset called ARTIC R2 or R3. I have only one.

There is 3 generations of Kineis modulation which are called :

- Argos 2 : uplink BPSK 200 Hz
- Argos 3 :
- Argos 4 :

Satellites passes on September 14th, 2026

In order to know the time when a Kineis satellite will be in our sky, you must connect the Kineis web server at:

<https://argos-system.cls.fr>

the login will be: iot2 and the password: tpiot2

Go in the “satellite pass prediction” menu.

Currently there is 5 active satellites and 3 which are inactives.

In order to get the time of the next passes you must select the correct dates and the correct location using latitude=47.5 longitude=6.8 and altitude=0.333 then click on the “simulate” button

Please, write the passes between 09:00 and 18:00 of today in the table below, only elevation upper to 30°.

The first one will be the satellite METOP-B (MB) from 11:29 to 11:41 local with maximum elevation 52° starting from North (8°) through West (290°) to South (213°).

Satellite	Start	End	Max elevation	beginning	Middle	Ending
MB	11:29 local	11:41 local	52°	North (8°)	West (290°)	South (213°)

For each pass, indicate which window is the best one for transmitting

MM22/C2xx : There is East windows (TD22 is West windows)

MM3D/C211 : There is North windows and South windows in the corridor

MM23/C212 : There is North and South windows

The best place outside is on the crossroad between the University Restaurant and UTBM building.

Hardware building – Configuring the jumpers on the KIM1 Board

Jumpers must be connected on the following PINs

To power the KIM1 board, there is 2 options from the USB port or through the microcontroller.

You have to configure the power from the Arduino microcontroller

// JP3 : 5V/Alim <--> Arduino

In order to limit the power consumption, it is necessary to allow the Arduino microcontroller to start and stop the power through a dedicated GPIO.

// JP7 : ON/OFF <--> Arduino

To communicate between the Arduino microcontroller and the KIM1 board there is an UART interface which is using one PIN to transmit and one PIN to receive with the satellite chipset.

// JP COM : TX <--> Arduino

// JP COM : RX <--> Arduino

Hardware building – Connecting the KIM1 board to the Adafruit Feather microcontroller

You need 5 wires to connect the 2 boards : Power and ground with 3 additional GPIO (ON/OFF, RX, TX).

// Adafruit Feather <----> KIM1 (Kineis Module)

// USB (5V) <----> 5V

// GND <----> GND

// GPIO5 <----> ON/OFF

// TX1 <----> RX_KIM1

// RX1 <----> TX_KIM1

Add your Kineis SMA antenna on the connector.

IMPORTANT : when you finished to connect the jumpers and link the PINs, ask the professor for a check before to power the overall. It is not allowed to connect the USB port alone.

The Adafruit Feather used for the practice is:

- ARM Cortex-M0 microcontroller + LoRa module: <https://www.adafruit.com/product/3178>

For building program, we use the development toolkit called Arduino with version 1.8.X where we add compilation components for our target hardware.

First Exercise on your own computer: Compile « blink » program

Download and install Arduino IDE 2.3.X

Then, install the plug-in to compile on the target architectures: Adafruit ARM-M0

To install the SAMD compiler for the ARM-M0 processor

Run Arduino tool then File → Preferences

https://adafruit.github.io/arduino-board-index/package_adafruit_index.json

(If there is already content, the URLs must be separated by a comma).

Tools → board type → board management

Install « Arduino SAMD boards (32-bit ARM cortex-M0+) »

then, install « Adafruit SAMD boards »

Choose the target hardware Tool → Board → « Adafruit Feather M0 express »

Test the blink example, to validate the correct installation of the compiler (File → Example)

Sometimes it can be needed to activate the programming mode doing a fast double-click on the RESET button of the Feather board and view the LED blinking in heartBeat mode.

Second practice: Sending a LoRa signal to the LoRa receiver in the room

Install the Radiohead library to use the LoRa/LoRaWAN communication modules by inserting the file .zip (croquis → include a library → zip file)

<http://www.airspayce.com/mikem/arduino/RadioHead/>

<http://www.airspayce.com/mikem/arduino/RadioHead/RadioHead-1.105.zip>

Creating the sending program

```
#include <SPI.h> // The LoRa module includes an interface with SPI bus
#include <RH_RF95.h> // Radiohead Library for LoRa communications
```

```
#define RFM95_CS 8
#define RFM95_RST 4
#define RFM95_INT 3
```

```
RH_RF95 rf95(RFM95_CS, RFM95_INT);
```

```
uint8_t buf[255];
uint8_t len = sizeof(buf);
```

```
void setup() {
  Serial.begin(115200);
```

```
  rf95.init();
  rf95.setModemConfig(RH_RF95::Bw125Cr45Sf128);
                                     // Bandwidth=125kHz Code rate 4/5 Spreading factor =2^7
  rf95.setFrequency(868.1); // 868.1, 868.2 or 868.3
  rf95.setTxPower(13, false); // +13dBm
}
```

```
void loop() {
  message="XXX KIM1 #"; // replace XXX with your name, in order to follow your work on the receiver
  message+=i;
  i++;
  message.toCharArray((char*)buf, message.length()+1);
  buf[message.length()+1] = 0;
  rf95.send((uint8_t *)buf, message.length()+2);
  rf95.waitPacketSent();
  Serial.print("Sent Message:");
  Serial.println(message);
  delay(2000);
}
```

Add a LED blink for each transmission, in order to visualise the time to send.

Third practice: Send a fixed message to the KINEIS satellite.

In order to send a message to the KIM1 chipset and then to the Kineis satellite, you need to add the following library :

- for green board : KIM by Kineis version 3.0.3 or 3.0.4
- for black board : KIM Arduino Library version 0.0.1

FOR GREEN BOARD

Then, load the example in File → Examples → KIM → KIM_example

if you compile this code for the Adafruit Feather M0 express it will not work, because this library has been done for Wemos D1 (ESP8266) and Arduino Uno.

We need to modify the KIM.h library which is in ~/Arduino/libraries/KIM/src/KIM.h adding the following lines before the “#endif” line:

```
#elif defined(ARDUINO_SAMD_ZERO) //< Adafruit Feather LoRa ARM Cortex M0

#define ON_OFF_KIM_PIN      5
#define RX_KIM              0
#define TX_KIM              1
#define softSerial          false
```

thanks to this modification, the correct GPIOs will be used. Save these changes.

Then, in the source code, replace the line:

```
#if defined(__AVR_ATmega4809__) // Arduino UNO Wifi Rev2
by
#if defined(ARDUINO_SAMD_ZERO)
```

Then cast each “const char” error with (char *)

Verify that your code is working by lighting up the red LED on the KIM1 for 8 seconds, then lighting up the green LED for 1 second, then lighting down both.

FOR BLACK BOARD

Then, load the example in File → Examples → KIM Arduino Library → Example_KIM1_SW2

Then we need to adapt the source code by removing the size table on each AT command :

```
char ID[] = "ID";
char FW[] = "FW";
char TX[] = "TX";
char PWR[] = "PWR";
```

Then, for each sizeof(string) you need to add (-1) on each sizeof(string) – 1 in order to work properly.

Verify that your code is working by lighting up the red LED on the KIM1 for 8 seconds, then lighting up the green LED for 1 second, then lighting down both.

Forth practice: Add LoRa transmit messages to follow your code

Add in your previous code, some LoRa sending messages including the data sent to the satellite, in order to follow the execution from the LoRa receiver.

Verify that, for each step of your code: before to send and after the result state of the send function, you transmit a LoRa message in order to monitor your code from inside on the receiver module. You can add a battery to power the hardware.

Put your hardware on a window or outside and wait for correct communication during a satellite pass.

You can control after 7 to 10 minutes on the Kineis Webserver that your message has been received by the satellite.

<https://argos-system.cls.fr>

go to the “view your data” to verify if your message has been received.

Connect the USB multimeter to measure the consumption during transmit time. Current=

Fifth practice: Deploy a LoRa network using a Kineis gateway

In a lonely area, we need to deploy 4 connected sensors. In order to do that, we use 3 LoRa sensors to centralize the data on the LoRa-Kineis gateway sensor. The gateway will send data of all the 4 sensors. For each Kineis message, there is 7 bytes of data for each sensor. Write the corresponding source code to modify the 7 bytes of each sensor by its battery level and temperature.